

Maria Mini Language Compiler

Compiler Design Lab Assignment

Roll: 233015042

1. Introduction

This project implements a small compiler for the **Maria Mini Language**. The compiler converts source programs into an executable backend representation through several compiler phases.

The implemented pipeline is:

Lexer → Parser/AST → Semantic Analysis → Code Generation → Three Address Code (TAC) → Optimization → Backend Execution

The main goal of this project is to demonstrate how a compiler analyzes, translates, optimizes, and executes a simple programming language.

2. Design Decisions

Lexer

The lexer recognizes keywords, identifiers, numbers, strings, operators, punctuation, and Boolean values. The language uses yes and no for logical values.

Parser

A **recursive descent parser** was used because the grammar is small and easy to understand and extend. Operator precedence is handled through separate parsing levels.

Abstract Syntax Tree (AST)

The parser creates an AST containing nodes for variable declarations, assignments, conditions, loops, print statements, literals, identifiers, unary expressions, and binary expressions.

Semantic Analysis

A symbol table is used to store declared variables and their types. The semantic analyzer checks:

- Undeclared variables
- Variable redeclaration
- Type mismatch
- Invalid operators
- Expression compatibility

Three Address Code (TAC)

Complex expressions are converted into TAC using temporary variables. Labels and jump instructions are used for conditions, loops, and short-circuit logical operations.

Optimization

The optimizer performs **constant folding**. For example, an expression such as:

$2 + 3$

can be changed to:

5

Constant values are also propagated through simple assignments when possible.

Backend

The backend converts TAC into executable Python-style code. A **program counter** is used to execute instructions and handle labels, conditional jumps, loops, and other control-flow operations.

3. Implemented Language Features

The project implements **Variant 3** features:

- String variables
- String concatenation using +

- do / during loop
- Short-circuit &&
- Short-circuit ||

Other supported features include:

- num variables
- decimal variables
- text variables
- logic variables
- Arithmetic operations
- Comparison operations
- Unary !
- check / otherwise conditional statements
- show statements
- Variable assignment

4. Testing

The project contains **17 Maria Mini Language test programs**.

- **12 valid test programs**
- **5 invalid test programs**

The valid tests cover strings, concatenation, arithmetic, logical operations, conditions, do-while loops, comparisons, unary operations, nested conditions, variables, optimization, and short-circuit behavior.

The invalid tests check:

- Undeclared variables
- Variable redeclaration
- Type mismatch
- Invalid expressions

Automated Python tests were also created for:

- Parser
- Semantic Analyzer

- Code Generator
- TAC Generator
- Backend

The final automated test result was:

5 passed in 0.14s

This confirms that the main compiler components and backend execution were working correctly for the implemented test cases.

5. Bug Encountered and Debugging

One important bug occurred during backend execution. Programs containing `if/else` and `do/during` control flow initially produced an **"invalid syntax"** error.

The initial backend represented jump operations using assignments such as:

```
goto_L1 = True
```

However, Python could not treat this as an actual `goto` operation. Therefore, control flow could not be executed correctly.

To debug the problem, the generated backend code was inspected and the error was reproduced using conditional and loop test programs. Simple arithmetic programs worked correctly, which showed that the problem was related to control flow.

The backend was then redesigned to use a **program counter**. Labels were mapped to instruction positions, and jump instructions changed the value of the program counter.

After this change, the `do-while` test successfully produced:

```
1
2
3
```

The conditional and logical tests also executed successfully.

6. What I Would Change

If more development time were available, I would improve the optimizer by adding:

- Stronger constant propagation
- Dead-code elimination
- Algebraic simplification
- More advanced expression optimization

I would also add more automated tests for nested conditions, loops, expressions, and combinations of different language features.

7. Conclusion

The Maria Mini Language Compiler demonstrates the major stages of compiler construction, starting from lexical analysis and parsing and continuing through semantic analysis, code generation, TAC generation, optimization, and backend execution.

The project successfully implements the required Variant 3 features, including strings with concatenation, `do/during` loops, and short-circuit logical operators. Testing was performed using both valid and invalid programs as well as automated Python tests.

Overall, the project provides a practical implementation of the fundamental concepts of compiler design.